



**UNIVERSIDAD AUTÓNOMA DEL ESTADO
DE HIDALGO**

FACULTAD DE INGENIERÍA

Construcción de AFD

DANIELA LOPEZ MTZ

PRACTICA 3

PROFESOR: EDUARDO CORNEJO VELAZQUEZ

GRUPO: 6-3

Práctica: Simulador de Autómata Finito Determinista (AFD)

Los Autómatas Finitos son máquinas teóricas que van cambiando de estado dependiendo de la entrada que reciben. La salida se limita a dos valores: aceptado y no aceptado, para indicar si la cadena que se ha recibido como entrada es o no válida.

Un Autómata Finito Determinista (AFD) se define como una tupla:

$$AFD = (\Sigma, Q, \delta, q_0, F)$$

- Σ es el alfabeto de entrada.
- Q es el conjunto finito y no vacío de los estados.
- δ es la función de transición, $\delta : Q \times \Sigma \rightarrow Q$.
- $q_0 \in Q$ es el estado inicial.
- $F \subset Q$ es un conjunto de estados finales de aceptación ($F \neq \emptyset$).

Sea $A = (\Sigma, Q, \delta, q_0, F)$ un AFD y sea $w = w_0w_1 \dots w_{n-1}$ una cadena de símbolos donde $w_i \in \Sigma$. Entonces, A acepta w si existe una secuencia de estados $r_0, r_1, \dots, r_n \in Q$ con tres condiciones:

1. $r_0 = q_0$
2. $\delta(r_i, w_i) = r_{i+1}$ para $i = 0, 1, \dots, n-1$
3. $r_n \in F$

La condición 1 establece que la máquina comienza en el estado inicial.

La condición 2 establece que la máquina cambia desde un estado hacia otro estado de acuerdo a la función de transición.

La condición 3 establece que la máquina acepta la cadena de entrada si termina en un estado de aceptación.

Entonces, A reconoce el lenguaje L si $L = \{w \mid A \text{ acepta } w\}$.

3. OBJETIVO GENERAL

Construir un programa en un lenguaje de programación de alto nivel para implementar un Autómata Finito Determinista (AFD) que permita analizar un conjunto de cadenas de símbolos (W) para determinar si pertenecen o no al lenguaje (L) reconocido por el AFD.

4. OBJETIVOS ESPECÍFICOS

- Identificar los componentes de un Autómata Finito Determinista (AFD).
- Implementar un AFD en un lenguaje de programación de alto nivel.
- Determinar si diferentes conjuntos de símbolos (W) pertenecen al lenguaje (L) reconocido por el AFD.

5. EQUIPO Y SOFTWARE A UTILIZAR

Equipo de cómputo con software para programación a elección de los alumnos.

6. PROCEDIMIENTO PARA EL DESARROLLO DE LA PRÁCTICA

Utilizar un lenguaje de programación de alto nivel para construir un programa que implemente un AFD con las siguientes condiciones:

Entrada

- La primera línea de entrada contiene seis enteros N, S, D, q_0, T y C ($1 \leq N, S, C \leq 100$, $1 \leq D \leq 100$, $1 \leq q_0 \leq N$, $0 \leq T \leq N$), donde $N = |Q|$, $S = |\Sigma|$, $D = N \times S$ es el número de transiciones en el autómata, q_0 es el estado inicial, $T = |F|$, y C es la cantidad de cadenas de símbolos a verificar si son aceptadas o no por el autómata M .
- La segunda línea contiene el alfabeto Σ , representado por una secuencia de S símbolos separados por espacios.
- La tercera línea contiene el conjunto de estados de aceptación F , representado por una secuencia de T enteros t_i ($1 \leq t_i \leq N$) separados por espacios.
- Las siguientes D líneas especifican las transiciones del autómata ($\delta(I, X) = J$).
- Finalmente, cada una de las siguientes C líneas contienen una cadena W .

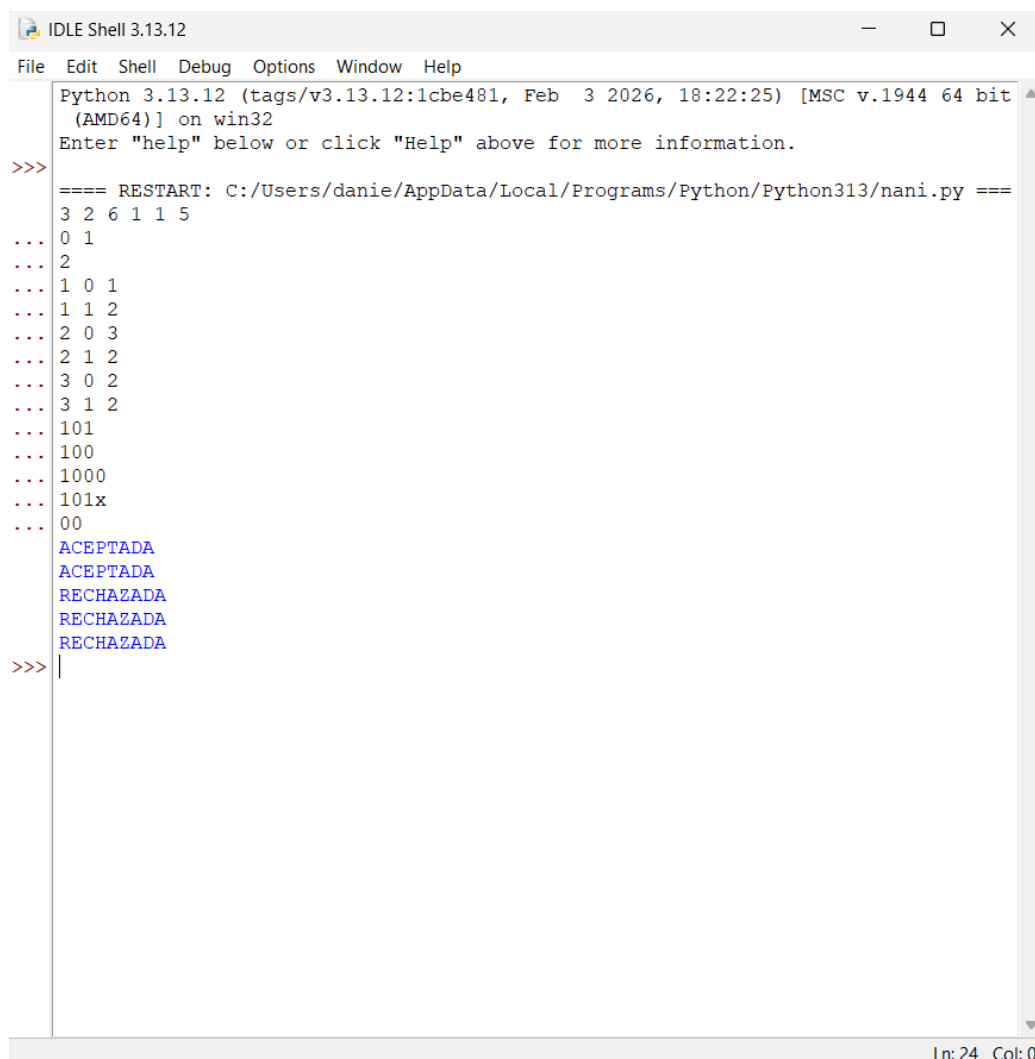
Salida

Para cada una de las C cadenas W se deberá imprimir el mensaje **ACEPTADA** si el autómata M acepta la cadena W , o **RECHAZADA** en caso contrario.

Código Fuente del Programa

```
1 import sys
2
3 def simular_afd():
4     linea1 = sys.stdin.readline().split()
5     if not linea1:
6         return
7     N, S, D, q0, T, C = map(int, linea1)
8
9     alfabeto = set(sys.stdin.readline().split())
10
11     linea_f = sys.stdin.readline().split()
12     estados_finales = set(map(int, linea_f)) if linea_f else set()
13
14     transiciones = {}
15     for _ in range(D):
16         partes = sys.stdin.readline().split()
17         if partes:
18             origen = int(partes[0])
19             simbolo = partes[1]
20             destino = int(partes[2])
21             transiciones[(origen, simbolo)] = destino
22
23     for _ in range(C):
24         cadena = sys.stdin.readline().rstrip('\r\n')
25
26         estado_actual = q0
27         cadena_valida = True
28
29         for simbolo in cadena:
30             if simbolo in alfabeto and (estado_actual, simbolo)
31             in transiciones:
32                 estado_actual = transiciones[(estado_actual,
33                 simbolo)]
34             else:
35                 cadena_valida = False
36                 break
37
38         if cadena_valida and estado_actual in estados_finales:
39             print("ACEPTADA")
40         else:
41             print("RECHAZADA")
42
43 if __name__ == "__main__":
44     simular_afd()
```

Evidencia de Ejecución



```
Python 3.13.12 (tags/v3.13.12:1cbe481, Feb  3 2026, 18:22:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
==== RESTART: C:/Users/danie/AppData/Local/Programs/Python/Python313/nani.py ====
3 2 6 1 1 5
... 0 1
... 2
... 1 0 1
... 1 1 2
... 2 0 3
... 2 1 2
... 3 0 2
... 3 1 2
... 101
... 100
... 1000
... 101x
... 00
... ACEPTADA
... ACEPTADA
... RECHAZADA
... RECHAZADA
... RECHAZADA
>>> |
```

Figura 1: Captura de pantalla .

7. CUESTIONARIO

1. ¿Qué paradigma de programación utilizaste para la implementación del AFD?, ¿Cuáles elementos del lenguaje de programación utilizaste?

Se utilizó el paradigma **imperativo y estructurado**, organizando la lógica mediante funciones y control de flujo secuencial. Los elementos clave del lenguaje Python empleados fueron lecturas de la entrada estándar (`sys.stdin.readline`), estructuras de control (`for`, `if-else`, `break`), funciones integradas de mapeo y conversión de datos (`map`, `int`, `split`, `set`), así como tuplas utilizadas como claves.

2. ¿Qué estructuras de datos utilizaste para implementar los componentes del AFD?

- **Diccionario (dict):** Representa la función de transición δ . La clave es una tupla (`estado_origen`, `simbolo`) y el valor es el `estado_destino`, permitiendo búsquedas en tiempo constante $O(1)$.
- **Conjunto (set):** Utilizado para el alfabeto Σ y los estados finales F , lo que agiliza la validación de pertenencia.
- **Cadenas de texto (str) y Enteros (int):** Para la manipulación de las palabras W y los identificadores de los estados de Q .

3. **¿Qué elementos del lenguaje de programación te permitieron implementar las funciones de transición?, ¿cuáles son las limitaciones del programa que construiste?**

Las funciones de transición se implementaron asociando una **tupla de dos elementos** como clave dentro de un diccionario.

Limitaciones:

- Está diseñado exclusivamente para autómatas finitos **deterministas** (AFD); no soporta no-determinismo (AFND) ni transiciones nulas (ϵ).
- Asume que las cadenas de entrada siguen estrictamente el formato especificado en las líneas de texto.

4. **¿Qué condiciones se deben cumplir para que el programa pueda evaluar cualquier AFD?**

- Que el autómata ingresado sea totalmente determinista (no exista más de un estado destino para el mismo par estado-símbolo).
- Que las líneas de entrada respeten la estructura exacta especificada en los parámetros (N, S, D, q_0, T, C) .
- Que los identificadores de los estados pertenezcan al rango $1 \leq q \leq N$.